

**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана**
**(национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

# Отчёт по лабораторной работе №1 по дисциплине «Анализ алгоритмов»

**Тема** Сравнительный анализ ресурсной эффективности алгоритмов поиска в массиве

**Студент** Скерджева А.М.

**Группа** ИУ7-54Б

**Преподаватель** Волкова Л.Л.

Москва, 2026

# РЕФЕРАТ

Отчёт содержит 27 с., 4 рис., 3 табл., 2 источника.

ПОИСК В МАССИВЕ, ЛИНЕЙНЫЙ ПОИСК, БИНАРНЫЙ ПОИСК, РЕКУРСИЯ, ЧИСЛО СРАВНЕНИЙ, ДОПОЛНИТЕЛЬНАЯ ПАМЯТЬ

Объект исследования — алгоритмы поиска целого числа в массиве. Цель работы — сравнительный анализ ресурсной эффективности линейного, двух нерекурсивных бинарных и рекурсивного бинарного алгоритмов поиска.

Разработаны схемы четырёх алгоритмов, реализована программа на языке C, выполнено функциональное тестирование. Временная эффективность оценена по числу сравнений искомого значения с элементами массива; дополнительная память оценена теоретически. Экспериментальные минимумы и максимумы числа сравнений получены поиском значений, которые присутствуют и отсутствуют в массиве.

Линейный поиск выполняет от одного до $N$ сравнений и не требует сортировки. В худшем случае число сравнений бинарных алгоритмов растёт логарифмически. Эти алгоритмы требуют упорядоченного массива. Рекурсивный вариант использует дополнительную память для стека вызовов. Алгоритм без ветвления уменьшает шаг поиска вдвое; число его сравнений зависит от размера массива и искомого значения.

# СОДЕРЖАНИЕ

ВВЕДЕНИЕ . . . . . 4
1 Аналитическая часть . . . . . 5
1.1 Постановка задачи . . . . . 5
1.2 Линейный поиск . . . . . 5
1.3 Классический нерекурсивный бинарный поиск . . . . . 5
1.4 Бинарный поиск без ветвления . . . . . 6
1.5 Рекурсивный бинарный поиск . . . . . 6
2 Конструкторская часть . . . . . 8
2.1 Требования к программе . . . . . 8
2.2 Структура программы . . . . . 8
2.3 Схемы алгоритмов . . . . . 9
3 Технологическая часть . . . . . 14
3.1 Язык программирования и инструменты сборки . . . . . 14
3.2 Реализация алгоритмов . . . . . 14
3.3 Учёт сравнений . . . . . 17
3.4 Тестирование . . . . . 18
4 Исследовательская часть . . . . . 19
4.1 Методика сравнения . . . . . 19
4.2 Число сравнений линейного поиска . . . . . 19
4.3 Число сравнений классического бинарного поиска . . . . . 20
4.4 Число сравнений бинарного поиска без ветвления . . . . . 20
4.5 Число сравнений рекурсивного бинарного поиска . . . . . 22
4.6 Дополнительная память . . . . . 22
4.7 Экспериментальные результаты . . . . . 23
4.8 Сопоставление алгоритмов . . . . . 24
ЗАКЛЮЧЕНИЕ . . . . . 26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . . 27

# ВВЕДЕНИЕ

Линейный и бинарный поиск решают одну задачу, но различаются числом сравнений, дополнительной памятью и требованиями к массиву. В работе сравниваются четыре реализации: линейная, две нерекурсивные бинарные и рекурсивная бинарная.

Цель работы — выполнить сравнительный анализ ресурсной эффективности алгоритмов поиска в массиве.

Для достижения цели необходимо решить следующие задачи:

— описать задачу поиска значения в массиве;
— разработать линейный, два нерекурсивных бинарных и рекурсивный бинарный алгоритмы;
— представить разработанные алгоритмы схемами;
— сформулировать требования к программному обеспечению;
— выбрать язык программирования и инструменты сборки;
— реализовать алгоритмы и программу для их сравнения;
— провести тестирование реализаций;
— оценить дополнительную память каждого алгоритма;
— оценить число сравнений в лучшем и худшем случаях;
— представить результаты сравнительного анализа в таблицах;
— сформулировать выводы о числе сравнений, затратах памяти и ограничениях алгоритмов.

# 1 Аналитическая часть

## 1.1 Постановка задачи

Дан массив целых чисел $a$ длины $N$ и искомое значение $x$. Нужно вернуть индекс элемента, равного $x$, или $-1$, если такого элемента нет. Индексы начинаются с нуля. При повторяющихся значениях допустим индекс любого равного элемента. Для пустого массива результат равен $-1$.

Линейный алгоритм получает исходный массив. Бинарные алгоритмы получают его копию, отсортированную по неубыванию. Линейный поиск возвращает индекс в исходном массиве, а бинарные — в отсортированной копии.

## 1.2 Линейный поиск

Линейный поиск последовательно проверяет элементы с индексами от нуля до $N - 1$ и завершает работу при первом равенстве. Если проверены все элементы и равенство не обнаружено, возвращается $-1$.

Линейный поиск не требует сортировки массива [1].

## 1.3 Классический нерекурсивный бинарный поиск

Область поиска задаётся полуинтервалом $[L, R)$. На каждом шаге вычисляется индекс среднего элемента. Если этот элемент меньше $x$, левая граница переносится за него. Если он больше $x$, правая граница становится равной его индексу. При равенстве возвращается индекс среднего элемента.

Благодаря упорядоченности массива исключённая часть не содержит искомого значения. После каждого неудачного шага остаётся не более половины элементов предыдущего участка с округлением вниз.

### 1.4 Бинарный поиск без ветвления

Второй нерекурсивный алгоритм взят из статьи М. Скарупке [2]. Поиск выполняется с шагом `step`, который последовательно уменьшается вдвое. Для выбора начального шага используются две функции: `bit_floor(N)` возвращает наибольшую степень двойки, не превышающую $N$, а `bit_ceil(N)` — наименьшую степень двойки, не меньшую $N$. Например, для $N = 10$ это 8 и 16 соответственно.

Сначала `begin` равен нулю, а `step` равен `bit_floor(N)`. Если размер массива не является степенью двойки, алгоритм сравнивает элемент с индексом `step` с искомым значением. Если этот элемент меньше искомого, поиск переносится в конец массива. Для оставшейся части заново выбирается шаг с помощью `bit_ceil`. Если за проверенным элементом больше ничего нет, возвращается $-1$.

В основном цикле шаг каждый раз делится на два. Проверяется элемент с индексом `begin + step`. Если он меньше искомого значения, `begin` увеличивается на `step`; иначе `begin` остаётся прежним. Цикл заканчивается, когда шаг становится равным нулю.

После цикла выполняется ещё одно сравнение элемента с индексом `begin` с искомым значением. Оно определяет позицию первого элемента, который не меньше искомого. В статье алгоритм возвращает эту позицию, даже если самого значения в массиве нет. В данной программе дополнительно проверяется равенство: если значение найдено, возвращается его индекс, иначе — $-1$.

Название «без ветвления» относится к машинному коду основного цикла. В исходном алгоритме есть условие `if`, которое GCC может преобразовать в инструкцию условного перемещения. Поэтому отсутствие условного перехода зависит от компилятора и параметров сборки; оно не означает отсутствие `if` в исходном коде.

### 1.5 Рекурсивный бинарный поиск

На каждом вызове проверяется размер текущего массива. При нулевом размере возвращается $-1$. Иначе сначала проверяется равенство среднего эле-

мента искомому значению. Если значения различаются, выбирается левая часть массива, когда искомое значение меньше среднего элемента, и правая — когда больше.

При поиске в правой части функция возвращает индекс относительно этой части. Чтобы получить индекс в исходном массиве, к нему прибавляется число элементов, оставшихся слева. При поиске в левой части индекс не меняется. Каждый следующий вызов получает меньше элементов, поэтому поиск заканчивается. В каждом вызове выбирается только одна часть массива, и рекурсивные вызовы образуют цепочку.

**Вывод**

Сформулирована задача поиска и рассмотрены четыре алгоритма её решения. Линейный поиск применим к неупорядоченным данным, а бинарные алгоритмы требуют предварительного упорядочивания. Два нерекурсивных бинарных алгоритма различаются порядком сравнений и возможностью раннего завершения.

# 2 Конструкторская часть

## 2.1 Требования к программе

Программа должна обеспечивать ввод размера массива и его элементов, сохранение исходного массива и подготовку отсортированной копии. При вводе размера, равного нулю, используется массив по умолчанию из десяти нечётных положительных чисел.

Программа служит испытательным стендом для сравнения алгоритмов. После подготовки данных она должна выводить меню и принимать искомое значение. В меню выбирается отдельный алгоритм или режим сравнения всех четырёх алгоритмов. Результатом поиска является индекс либо сообщение об отсутствии элемента. Для выбранного алгоритма указывается, к какому массиву относится индекс. Режим сравнения должен выводить индекс и число сравнений для каждого алгоритма.

Основные функции поиска должны возвращать только индекс. Расширенные функции для исследования должны дополнительно возвращать число выполненных сравнений искомого значения с элементами массива.

## 2.2 Структура программы

Программа состоит из модуля подготовки массивов, четырёх модулей алгоритмов поиска, модуля меню, диспетчера и точки входа. Функция `main` создаёт и освобождает исходный массив и его отсортированную копию. Диспетчер передаёт оба массива модулю меню. Для линейного поиска выбирается исходный массив, для остальных алгоритмов — отсортированная копия.

Каждая функция поиска получает указатель на массив, его размер и искомое значение. Функции поиска не меняют значения элементов и их порядок. Для бинарного поиска требуется неубывающий порядок. Расширенная функция возвращает структуру с полями `index` и `comparisons`.

### 2.3 Схемы алгоритмов

В схемах используются имена параметров и переменных из исходного кода: `arr` — массив, `size` — его размер, `standart` — искомое значение. Деление целочисленное. Функция `compare(a, b)` возвращает результат сравнения $a < b$. В составных условиях вторая часть проверяется только тогда, когда первая истинна. Рекурсивная функция возвращает индекс относительно переданного подмассива; при поиске в правой части к нему прибавляется `median + 1`.

Схема линейного поиска представлена на рисунке 1.

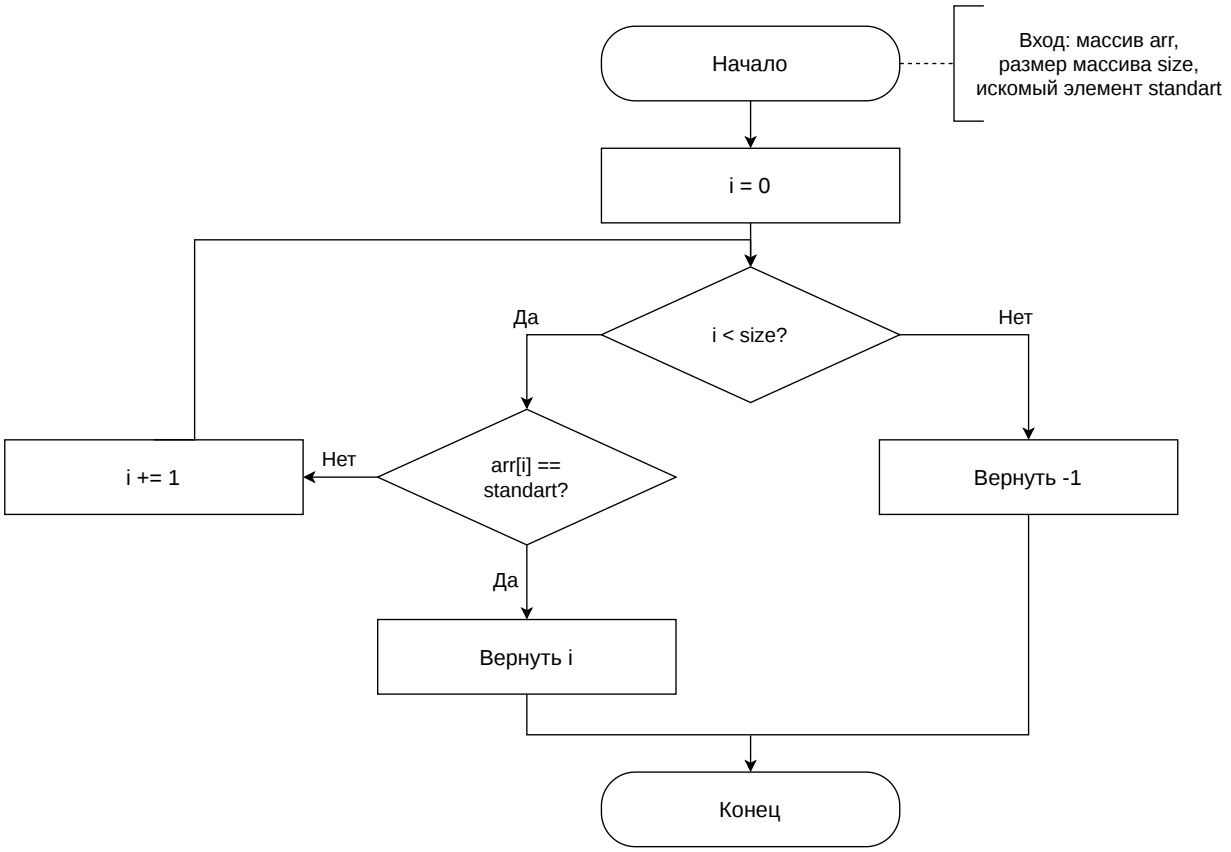



Рисунок 1 — Схема линейного поиска

Схема классического нерекурсивного бинарного поиска представлена на рисунке 2.

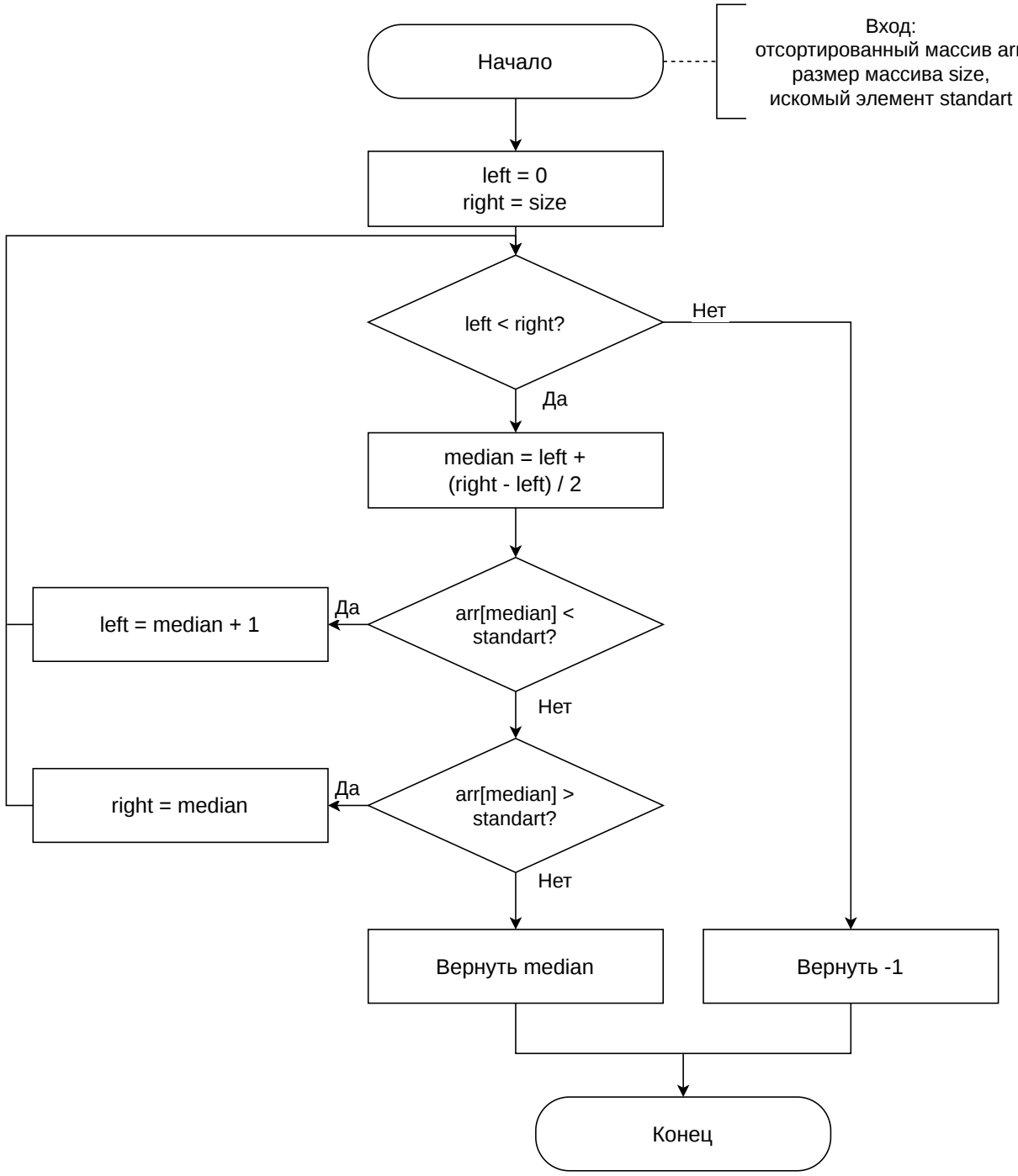



Рисунок 2 — Схема классического нерекурсивного бинарного поиска

Схема бинарного поиска без ветвления представлена на рисунке 3.

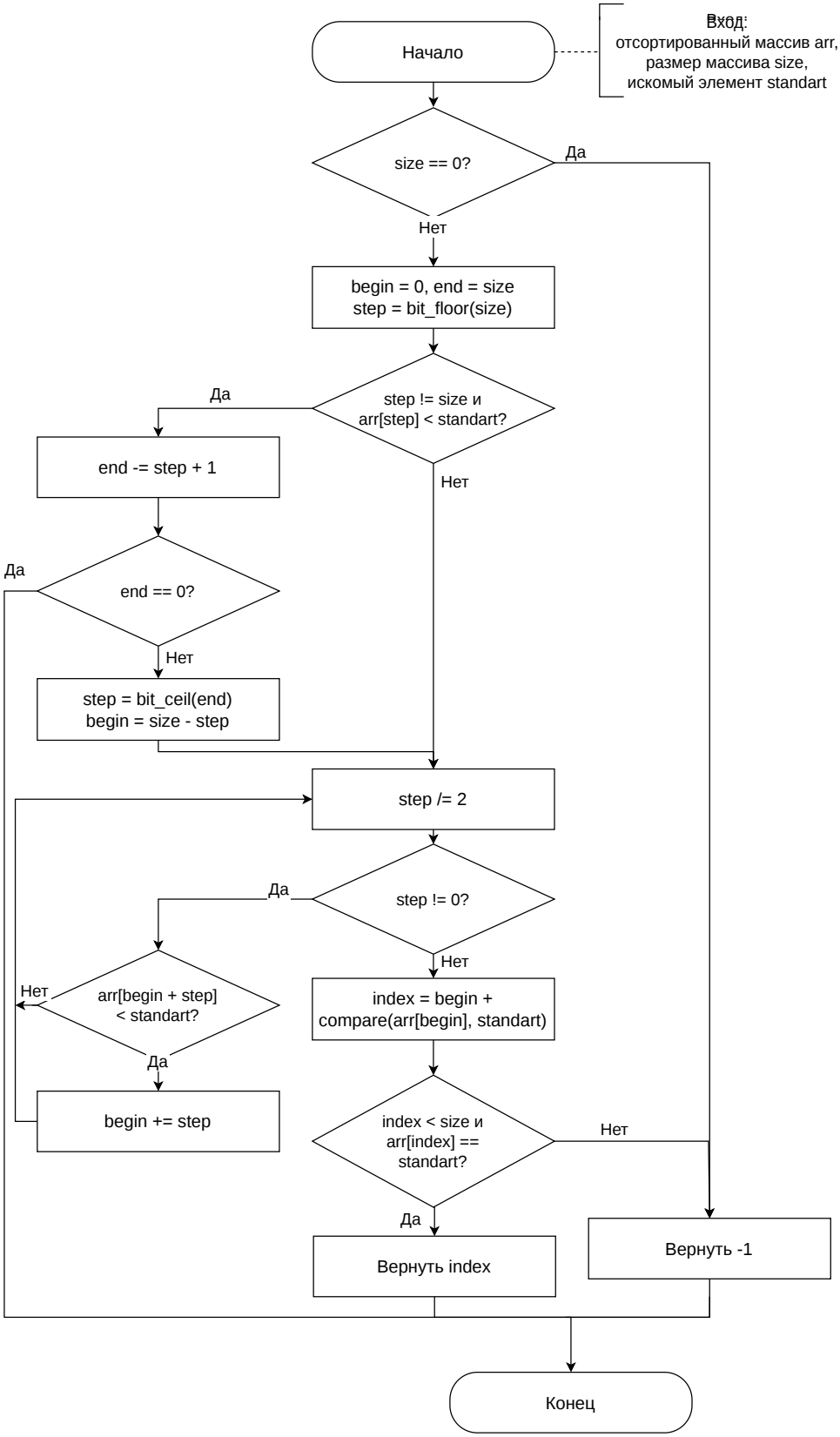



Рисунок 3 — Схема бинарного поиска без ветвления

Схема рекурсивного бинарного поиска представлена на рисунке 4.

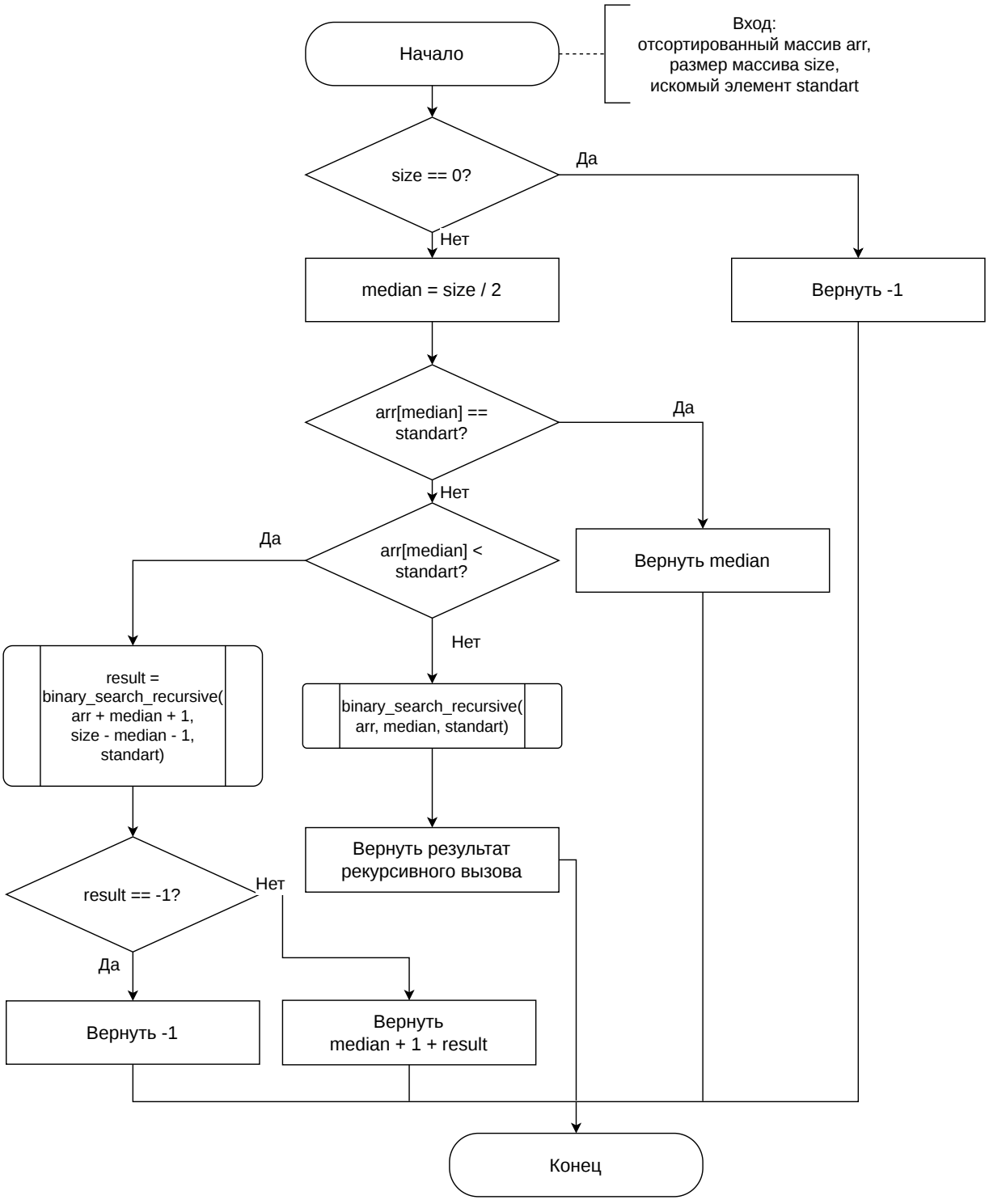



Рисунок 4 — Схема рекурсивного бинарного поиска

**Вывод**

Разработаны схемы четырёх функций поиска. Для сравнения алгоритмов предусмотрены отдельные версии функций, которые дополнительно подсчитывают число сравнений.

# 3 Технологическая часть

## 3.1 Язык программирования и инструменты сборки

Для реализации выбран язык C: он имеет статическую типизацию, поддерживает работу с массивами, указателями и рекурсивными функциями. Проверка выполнена компилятором GCC 16.1.1 с параметрами `-std=c11 -Wall -Wextra -Werror -Wpedantic`. Основная программа собирается с помощью CMake. Типы `size_t` и `ssize_t` используются для размера массива и результата поиска соответственно.

Подготовка отсортированной копии выполняется пузырьковой сортировкой вне функций поиска. Её затраты рассматриваются отдельно от числа сравнений одного поискового запроса. Элементы массива имеют тип `int`.

## 3.2 Реализация алгоритмов

Основные функции поиска приведены в листингах 1–4. В алгоритме без ветвления и рекурсивной реализации вспомогательная функция `compare(a, b)` возвращает результат сравнения `a < b`.

Листинг 1 — Линейный поиск

```
#include "search_algorithms.h"

ssize_t linear_search(const int *arr, size_t size, int standart)
{
    for (size_t i = 0; (i < size); ++i)
    {
        if (arr[i] == standart)
            return i;
    }
    return -1;
}
```

Листинг 2 — Классический нерекурсивный бинарный поиск

```
#include "search_algorithms.h"

ssize_t binary_search_classic(const int *arr, size_t size, int
   standart)
{
    size_t left = 0, right = size;
    while (left < right)
    {
        size_t median = left + (right - left) / 2;
        if (arr[median] < standart)
            left = median + 1;
        else if (arr[median] > standart)
            right = median;
        else
            return median;
    }
    return -1;
}
```

Листинг 3 — Бинарный поиск без ветвления

```
#include "search_algorithms.h"

size_t bit_floor(size_t size);
size_t bit_ceil(size_t size);
int compare(int a, int b);

ssize_t binary_search_branchless(const int *arr, size_t size, int
    standart)
{
    if (!size)
        return -1;

    size_t begin = 0, end = size;
    size_t step = bit_floor(size);
    if (step != size && compare(arr[step], standart))
    {
        end -= step + 1;
        if (!end)
            return -1;

        step = bit_ceil(end);
        begin = size - step;
    }

    for (step /= 2; step != 0; step /= 2)
        if (compare(arr[begin + step], standart))
            begin += step;

    size_t index = begin + compare(arr[begin], standart);
    return index < size && arr[index] == standart ? (ssize_t)
   index : -1;
}
```

Листинг 4 — Рекурсивный бинарный поиск

```
#include "search_algorithms.h"

static int compare(int a, int b);

ssize_t binary_search_recursive(const int *arr, size_t size, int
   standart)
{
    if (!size)
        return -1;

    size_t median = size / 2;

    if (arr[median] == standart)
        return (ssize_t)median;

    if (compare(arr[median], standart))
    {
        ssize_t result = binary_search_recursive(arr + median +
   1, size - median - 1, standart);
        if (result == -1)
            return -1;
        return (ssize_t)(median + 1) + result;
    }

    return binary_search_recursive(arr, median, standart);
}
```

### 3.3 Учёт сравнений

Счётчик учитывает каждое выполненное сравнение искомого значения с элементом массива. Каждое выполненное сравнение с помощью ==, < или > увеличивает счётчик на единицу. Например, если сначала проверяется `a[M] < x`, а затем `a[M] > x`, учитываются два сравнения. В рекурсивном поиске складываются сравнения, выполненные во всех вызовах. Проверки границ и условий циклов, сортировка, ввод и вывод не учитываются.

### 3.4 Тестирование

В таблице 1 приведены тесты всех четырёх алгоритмов. Для линейного поиска используются исходные данные, для бинарных — отсортированная копия.

Таблица 1 — Функциональные тесты алгоритмов поиска

| № | Описание | Входные данные | Ожидаемый и полученный результат |
|---|---|---|---|
| 1 | Пустой массив | $a = []$, $x = 7$ | $-1$, сравнений нет |
| 2 | Единственный элемент найден | $a = [4]$, $x = 4$ | Индекс 0 |
| 3 | Значение меньше единственного | $a = [4]$, $x = 3$ | $-1$ |
| 4 | Значение больше единственного | $a = [4]$, $x = 5$ | $-1$ |
| 5 | Первый элемент | $a = [0, 2, 4, 6, 8]$, $x = 0$ | Индекс 0 |
| 6 | Средний элемент | $a = [0, 2, 4, 6, 8]$, $x = 4$ | Индекс 2 |
| 7 | Последний элемент | $a = [0, 2, 4, 6, 8]$, $x = 8$ | Индекс 4 |
| 8 | Отсутствие между элементами | $a = [0, 2, 4, 6, 8]$, $x = 3$ | $-1$ |
| 9 | Повторяющиеся значения | $a = [1, 1, 1, 2]$, $x = 1$ | Индекс из $\{0, 1, 2\}$ |
| 10 | Границы типа `int` | `a = [INT_MIN, 0, INT_MAX], x = INT_MAX` | Индекс 2 |
| 11 | Исходный порядок | $a = [8, 0, 4]$, $x = 8$ | Линейный: 0; бинарные: 2 |

Все тесты пройдены успешно.

### Вывод

Реализации четырёх алгоритмов прошли функциональные проверки.

# 4 Исследовательская часть

## 4.1 Методика сравнения

Временная эффективность оценивается числом выполненных сравнений искомого значения с элементами массива. Для каждого фиксированного размера лучший случай означает минимум, а худший — максимум по всем возможным запросам, включая отсутствие значения. Сравниваются строго возрастающие массивы. Все формулы ниже относятся к $N \geq 1$. При $N = 0$ число сравнений равно нулю. Записи $\lfloor z \rfloor$ и $\lceil z \rceil$ означают округление числа $z$ вниз и вверх до целого соответственно. Например, $\lfloor 5/2 \rfloor = 2$, а $\lceil 5/2 \rceil = 3$.

Эксперимент выполнен в среде Fedora Linux 44, на процессоре AMD Ryzen 5 5600H с 12 логическими процессорами. Доступный операционной системе объём оперативной памяти составляет около 14 ГиБ. Стенд подсчёта сравнений собран GCC 16.1.1 без оптимизации, с параметром `-O0`.

## 4.2 Число сравнений линейного поиска

При наличии значения первое равенство на позиции $i$ требует $i + 1$ сравнений. Если значение отсутствует, проверяются все $N$ элементов. Поэтому для непустого массива

$$C_{\text{lin}}^{\min}(N) = 1, \qquad C_{\text{lin}}^{\max}(N) = N. \tag{1}$$

где $C_{\text{lin}}^{\min}$ и $C_{\text{lin}}^{\max}$ — минимальное и максимальное число сравнений линейного поиска.

Лучший случай наступает при $x = a_0$. Худший — при отсутствии $x$ либо при первом появлении $x$ на последней позиции. Асимптотические оценки составляют $\Theta(1)$ и $\Theta(N)$ соответственно.

### 4.3 Число сравнений классического бинарного поиска

Каждая итерация содержит сравнение $a_M < x$. Если средний элемент не меньше искомого значения, проверяется $a_M > x$. Если искомое значение равно среднему элементу исходного массива, выполняется два сравнения. Для $N = 1$ и $N = 2$ поиск значения, большего всех элементов массива, заканчивается после одной итерации и одного сравнения. Для $N \geq 3$ результат не может определиться после одного сравнения. Получаем

$$C_{\text{cl}}^{\min}(N) = \begin{cases} 1, & N \leq 2, \\ 2, & N \geq 3, \end{cases} \qquad C_{\text{cl}}^{\max}(N) = 2\lfloor \log_2 N \rfloor + 2. \tag{2}$$

где $C_{\text{cl}}^{\min}$ и $C_{\text{cl}}^{\max}$ — минимум и максимум числа сравнений классического бинарного поиска.

Худший случай достигается при $x < a_0$: выбирается левая половина длины $\lfloor N/2 \rfloor$, и на каждой непустой итерации выполняются два сравнения. Так как число элементов на каждой итерации уменьшается примерно вдвое, для массива из $N$ элементов потребуется не более $\lfloor \log_2 N \rfloor + 1$ итераций. В лучшем случае оценка составляет $\Theta(1)$, в худшем — $\Theta(\log N)$.

### 4.4 Число сравнений бинарного поиска без ветвления

В основном цикле шаг уменьшается вдвое, и на каждой итерации выполняется одно сравнение. После цикла сравниваются элемент с индексом `begin` и искомое значение. Если полученная позиция находится внутри массива, выполняется ещё одна проверка на равенство. Подготовка поиска также может потребовать одного сравнения, если размер массива не является степенью двойки.

Если $N$ является степенью двойки, цикл выполняет $\log_2 N$ сравнений. Поэтому лучший случай требует $\log_2 N + 1$ сравнений, а худший — $\log_2 N + 2$. Лучший случай наступает, когда искомое значение больше всех элементов: полученная позиция находится за концом массива и проверка равенства не выполняется.

Для остальных размеров число итераций зависит от того, какая часть массива выбрана перед циклом. Обозначим через $p = \texttt{bit_floor}(N)$ наиболь-

шую степень двойки, не превышающую $N$. Худший случай достигается при значении, меньшем всех элементов: алгоритм остаётся в начале массива. Лучший — при значении, большем всех элементов: выбирается конечная часть массива. Если $N = p + 1$, поиск завершается сразу после первого сравнения. Иначе в конечной части остаётся $N - p - 1$ элементов, и шаг задаётся функцией `bit_ceil`.

Таким образом, для непустого массива получаем:

$$C_{\text{bl}}^{\min}(N) = \begin{cases} \log_2 N + 1, & N = p, \\ 1, & N = p + 1, \\ \lceil \log_2(N - p - 1) \rceil + 2, & N > p + 1, \end{cases}$$

$$C_{\text{bl}}^{\max}(N) = \begin{cases} \log_2 N + 2, & N = p, \\ \lfloor \log_2 N \rfloor + 3, & N > p. \end{cases} \tag{3}$$

где $C_{\text{bl}}^{\min}$ и $C_{\text{bl}}^{\max}$ — минимальное и максимальное число сравнений бинарного поиска без ветвления;
$p$ — наибольшая степень двойки, не превышающая $N$.

В худшем случае число сравнений имеет оценку $\Theta(\log N)$. Лучший случай зависит от размера: при $N = p + 1$ достаточно одного сравнения, а при размерах, равных степени двойки, число сравнений растёт логарифмически. Проверки и вычисления внутри `bit_floor` и `bit_ceil` не сравнивают искомое значение с элементами массива и в выбранный счётчик не входят.

### 4.5 Число сравнений рекурсивного бинарного поиска

Рекурсивная реализация начинает проверку непустого подмассива с равенства. Поэтому лучший случай, $x = a_{\lfloor N/2 \rfloor}$, требует одного сравнения. Если средний элемент не равен искомому значению, выполняются два сравнения: проверка равенства и проверка `a[M] < x`. В худшем случае таких вызовов будет $\lfloor \log_2 N \rfloor + 1$, после чего функция получит пустую часть массива и завершится. Следовательно,

$$C_{\text{rec}}^{\min}(N) = 1, \qquad C_{\text{rec}}^{\max}(N) = 2\lfloor \log_2 N \rfloor + 2. \tag{4}$$

где $C_{\text{rec}}^{\min}$ и $C_{\text{rec}}^{\max}$ — минимальное и максимальное число сравнений рекурсивного бинарного поиска.

Если значение найдено в $k$-м вызове, до этого выполнено $k-1$ вызовов с двумя сравнениями, а последний требует одного сравнения. Всего получается $2k - 1$. Если значение не найдено, каждый из $k$ вызовов с непустой частью массива требует двух сравнений, всего — $2k$. Асимптотически лучший случай имеет оценку $\Theta(1)$, а худший — $\Theta(\log N)$.

### 4.6 Дополнительная память

Память входного массива одинакова для всех алгоритмов и составляет $N \cdot \text{sizeof}(\texttt{int})$ байт. Она не включается в дополнительную память поиска. Линейный алгоритм хранит индекс; нерекурсивные бинарные алгоритмы хранят ограниченное число индексов, границ и временных значений. Их дополнительная память имеет оценку $\Theta(1)$.

При рекурсивном поиске в стеке сохраняются данные каждого активного вызова: параметры, локальные переменные и адрес возврата. Поскольку каждый вызов порождает не более одного следующего вызова, дерево рекурсивных вызовов представляет собой цепочку. Её высота равна числу одновременно активных вызовов. Эти данные образуют кадр вызова, размер которого не зависит от длины массива. В худшем случае после непустых участков вызывается функция для пустого участка. Поэтому

$$H_{\max}(N) = \lfloor \log_2 N \rfloor + 2. \tag{5}$$

где $H_{\max}$ — максимальное число одновременно активных вызовов, включая вызов для пустого участка.

В лучшем случае выполняется один вызов и требуется $\Theta(1)$ дополнительной памяти. В худшем случае число кадров растёт как $\log N$, поэтому расход памяти составляет $\Theta(\log N)$. Точный объём в байтах зависит от компилятора и архитектуры. Оценка относится к выполнению рекурсивных вызовов без их устранения оптимизатором.

Стенд дополнительно хранит отсортированную копию, что требует $\Theta(N)$ памяти. Это общая подготовка данных для бинарных алгоритмов, а не дополнительная память их функций. Пузырьковая сортировка в используемой программе выполняет $N(N-1)/2$ сравнений элементов и имеет трудоёмкость $\Theta(N^2)$; исходный порядок сохраняется. Затраты сортировки не включаются в таблицы стоимости одного поискового запроса. При изменении массива упорядоченность копии необходимо восстанавливать.

### 4.7 Экспериментальные результаты

Для каждого размера создан массив $a_i = 2i$, $0 \le i < N$. Поиск выполняется для каждого целого значения $x$ от $-1$ до $2N$ включительно. Такой набор охватывает присутствующие значения, каждый промежуток между соседними элементами и оба внешних диапазона. Поскольку поведение алгоритмов определяется отношениями порядка и равенства, этот перебор даёт точные минимум и максимум для выбранного массива, а не случайную выборку.

Результаты представлены в таблице 2. В каждой ячейке указаны минимум и максимум числа сравнений, разделённые косой чертой. Для пустого массива

все четыре функции выполняют ноль сравнений элементов с $x$.

Таблица 2 — Экспериментальные минимум и максимум числа сравнений

| $N$ | Линейный | Классический бинарный | Бинарный без ветвления | Рекурсивный бинарный |
|---|---|---|---|---|
| 1 | 1 / 1 | 1 / 2 | 1 / 2 | 1 / 2 |
| 2 | 1 / 2 | 1 / 4 | 2 / 3 | 1 / 4 |
| 3 | 1 / 3 | 2 / 4 | 1 / 4 | 1 / 4 |
| 8 | 1 / 8 | 2 / 8 | 4 / 5 | 1 / 8 |
| 10 | 1 / 10 | 2 / 8 | 2 / 6 | 1 / 8 |
| 16 | 1 / 16 | 2 / 10 | 5 / 6 | 1 / 10 |
| 32 | 1 / 32 | 2 / 12 | 6 / 7 | 1 / 12 |
| 64 | 1 / 64 | 2 / 14 | 7 / 8 | 1 / 14 |
| 128 | 1 / 128 | 2 / 16 | 8 / 9 | 1 / 16 |
| 256 | 1 / 256 | 2 / 18 | 9 / 10 | 1 / 18 |
| 1024 | 1 / 1024 | 2 / 22 | 11 / 12 | 1 / 22 |

### 4.8 Сопоставление алгоритмов

Итоговое сравнение приведено в таблице 3. Формулы справедливы для $N \geq 1$; условия относятся к строго возрастающему массиву. Дополнительная память указана для функции поиска без входного массива и отсортированной копии. В таблице $m = \lfloor N/2 \rfloor$ — индекс среднего элемента. Число сравнений алгоритма без ветвления определяется формулой (3).

Таблица 3 — Сравнение ресурсной эффективности алгоритмов поиска

| Алгоритм | Лучший случай и условие | Худший случай и условие | Ограничения | Память, худший случай |
|---|---|---|---|---|
| Линейный | 1; $x = a_0$ | $N$; $x$ отсутствует или впервые найден на позиции $N - 1$ | Нет требования упорядоченности | $\Theta(1)$ |
| Классический бинарный | 1 при $N \leq 2$, $x > a_{N-1}$; 2 при $N \geq 3$, $x = a_m$ | $2\lfloor \log_2 N \rfloor + 2$; $x < a_0$ | Массив по неубыванию | $\Theta(1)$ |
| Бинарный без ветвления | $C_{\text{bl}}^{\min}(N)$; $x > a_{N-1}$ | $C_{\text{bl}}^{\max}(N)$; $x < a_0$ | Массив по неубыванию | $\Theta(1)$ |
| Рекурсивный бинарный | 1; $x = a_m$ | $2\lfloor \log_2 N \rfloor + 2$; $x < a_0$ | Массив по неубыванию | $\Theta(\log N)$ |

При $N = 1024$ худший случай линейного поиска требует 1024 сравнений, классического и рекурсивного бинарных — 22, алгоритма без ветвления — 12. Однако для запроса, равного среднему элементу, рекурсивный алгоритм выполняет одно сравнение, классический — два, а алгоритм без ветвления — 12. Поэтому предпочтительность алгоритма зависит от сценария запросов.

**Вывод**

Для всех исследованных размеров число сравнений совпало с теоретическими оценками. В худшем случае бинарные алгоритмы требуют логарифмического числа сравнений, линейный — $N$. На исследованных размерах $N \geq 8$ алгоритм без ветвления выполняет меньше сравнений в худшем случае, чем классический и рекурсивный бинарные алгоритмы. Однако при поиске среднего элемента он не завершает цикл сразу после обнаружения равенства. Рекурсивный вариант имеет логарифмические затраты памяти. При выборе бинарного поиска нужно также учитывать предварительную сортировку массива.

# ЗАКЛЮЧЕНИЕ

Цель работы достигнута: выполнен сравнительный анализ ресурсной эффективности четырёх алгоритмов поиска в массиве.

В ходе работы решены поставленные задачи:

— описаны входные данные и результат поиска;
— разработаны линейный, два нерекурсивных бинарных и рекурсивный бинарный алгоритмы;
— построены схемы основных функций;
— сформулированы требования к программе;
— выбраны язык программирования и инструменты сборки;
— реализованы алгоритмы и программа для их сравнения;
— выполнено функциональное тестирование и проверка счётчиков;
— оценена дополнительная память, включая цепочку рекурсивных вызовов;
— получены оценки числа сравнений в лучшем и худшем случаях;
— результаты теоретического анализа и эксперимента представлены в таблицах;
— сформулированы выводы о ресурсных затратах и ограничениях алгоритмов.

Линейный поиск не требует упорядоченности и использует постоянную дополнительную память, но в худшем случае выполняет $N$ сравнений. Все рассмотренные бинарные алгоритмы требуют упорядоченного массива и имеют логарифмическое число сравнений в худшем случае. Классический и рекурсивный варианты могут завершить поиск сразу после обнаружения равенства. Алгоритм без ветвления сначала определяет позицию первого элемента, не меньшего искомого, а затем проверяет равенство.

Нерекурсивные реализации используют $\Theta(1)$ дополнительной памяти, рекурсивная — $\Theta(\log N)$ в худшем случае. Эксперимент подтвердил выведенные формулы на исследованных размерах. Время выполнения программ в этой работе не измерялось.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. — 4th ed. — Cambridge, MA : The MIT Press, 2022. — 1312 p. — ISBN 978-0-262-04630-5.

2. Skarupke M. Beautiful Branchless Binary Search // Probably Dance. — 2023. — 27 April. — URL: `https://probablydance.com/2023/04/27/beautiful-branchless-binary-search/` (дата обращения: 11.10.2026).